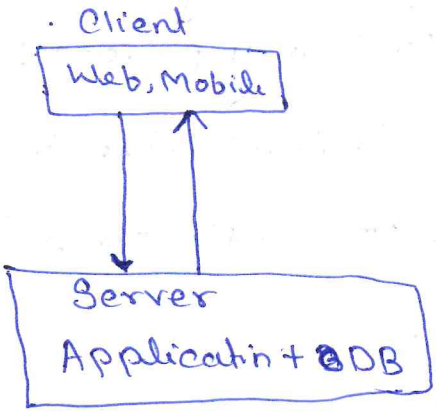
# Scale from Zero to MILLION

## Scale 0 to Million Users

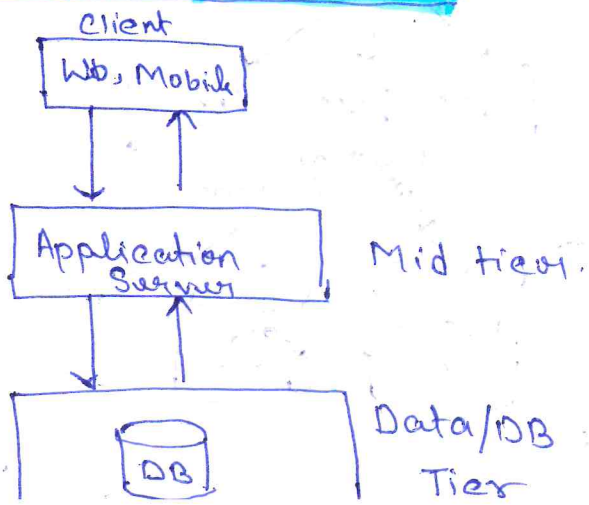
1. Single Server
2. Application & DB Server Separation
3. Load Balancer + Multiple App Servers.
4. Database Replication
5. CHCHE
6. CDN
7. Data Centre
8. Messaging Queue.
9. Database Scaling.

Concept && Coding

### 1. Single Server :→

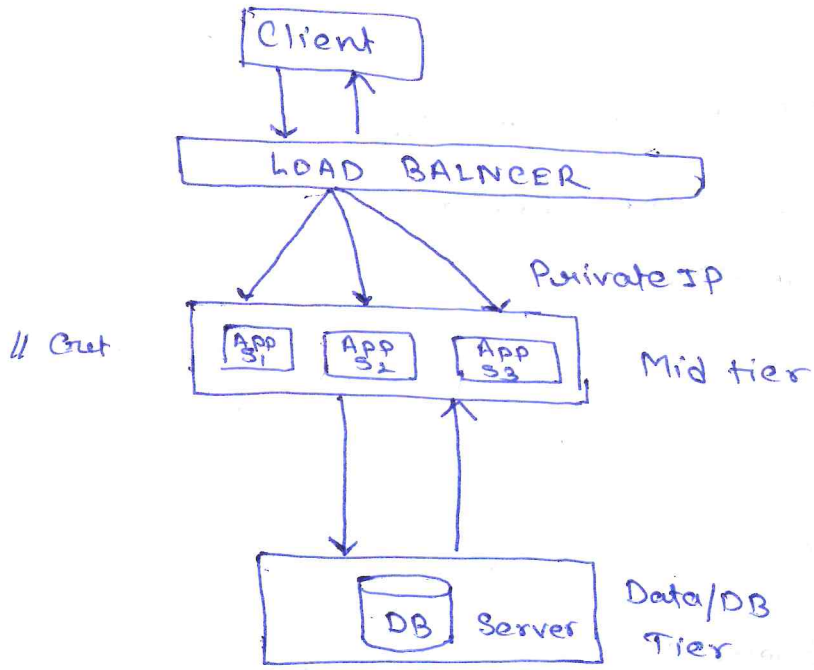



### 2. Application & DB Server Separation



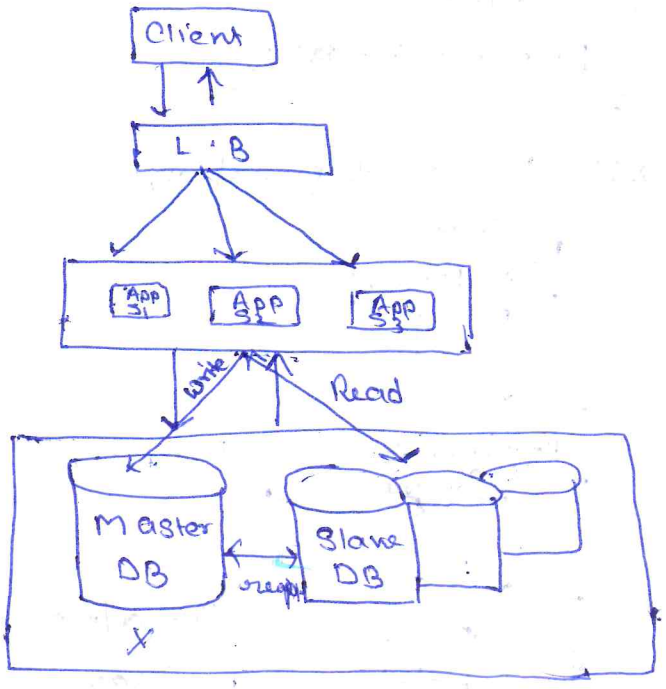
## 3: Load Balancer + Multiple app Servers

* Bringing the Load Balancer & Multiple App Server




- Client can ~~oty~~ only talk Load Balncer. Load Balancer can deside whom I want to farward my request.
- Load balancer ~~can~~ brings security and privacy. Load Balancer and App server can talk only private IP.
- Load balance can equaly devide the traffic request.

## 4. Database Replication :- (To handle the failure issue)

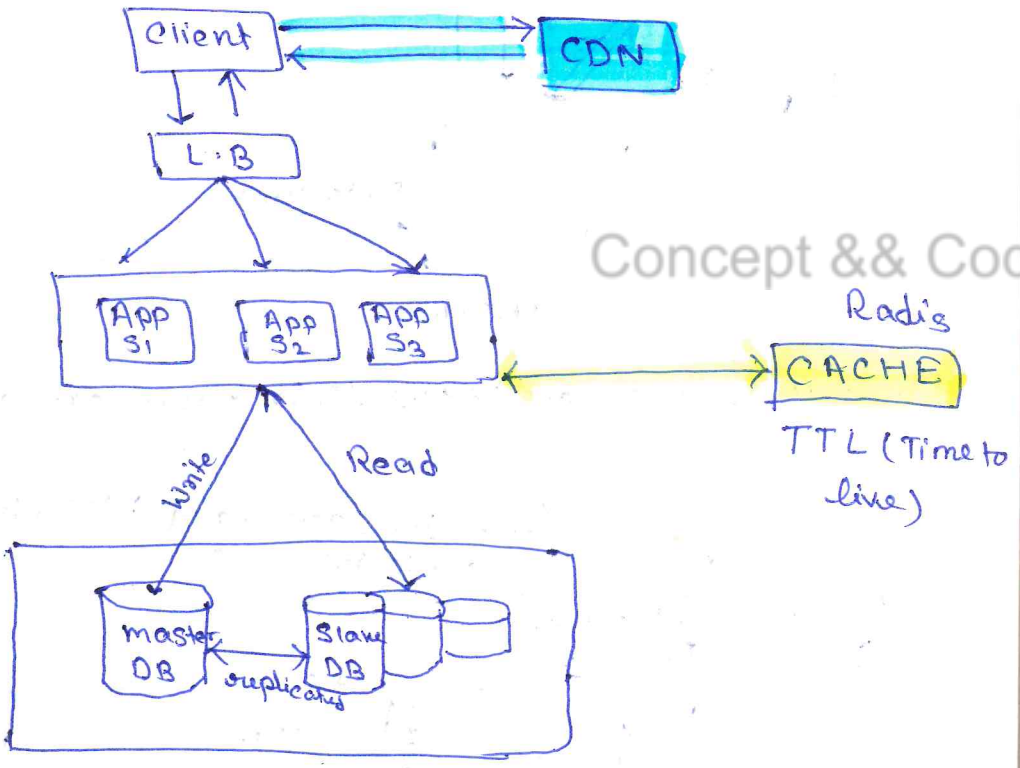



Concept && Coding

- Write request go on master.
- Read request go on Slave.
- Suppose in slave DB sometime due to damage. other node of DB is up. It should handle the request.
- Suppose If Master DB will be fail, then any slave DB can get promoted to MasterDB.

## 5. CHCHE (Performence improve)

Every time application Server talk with DB. Consider that is a Network call. It's take a lots of time. this is an expensive call.



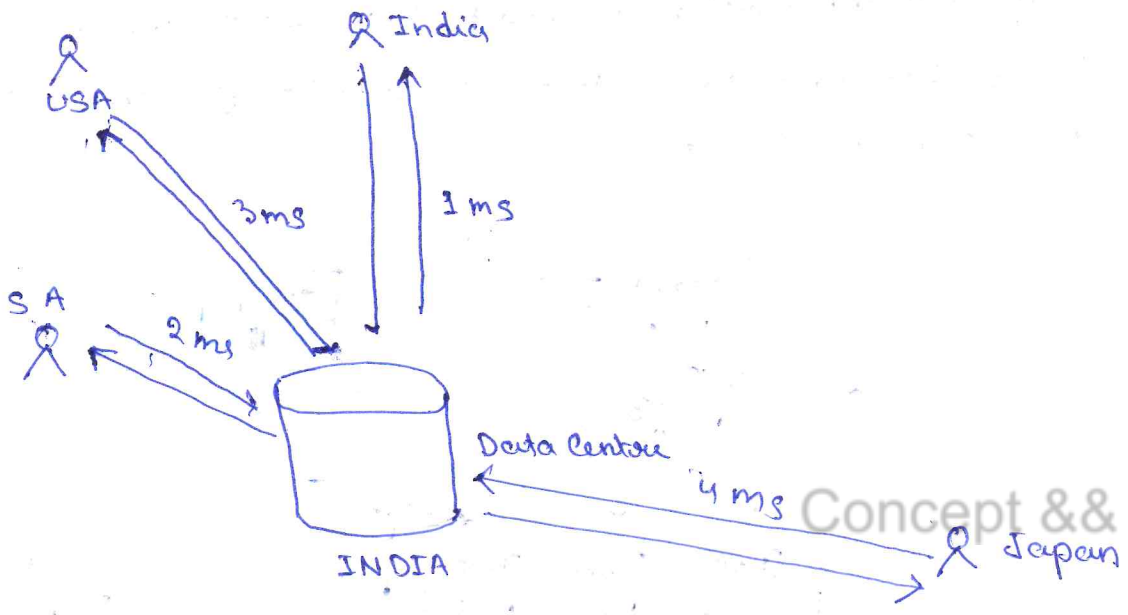

Concept && Coding

* Always before going to the DB, we should check on "CACHE". Data have present in CACHE or not. If Data will be present in CACHE we can consider as "Cachehit". If data will be not present in "CHCHE" that is called as "cacheMiss".

* Time to Live depending upon application requirement.

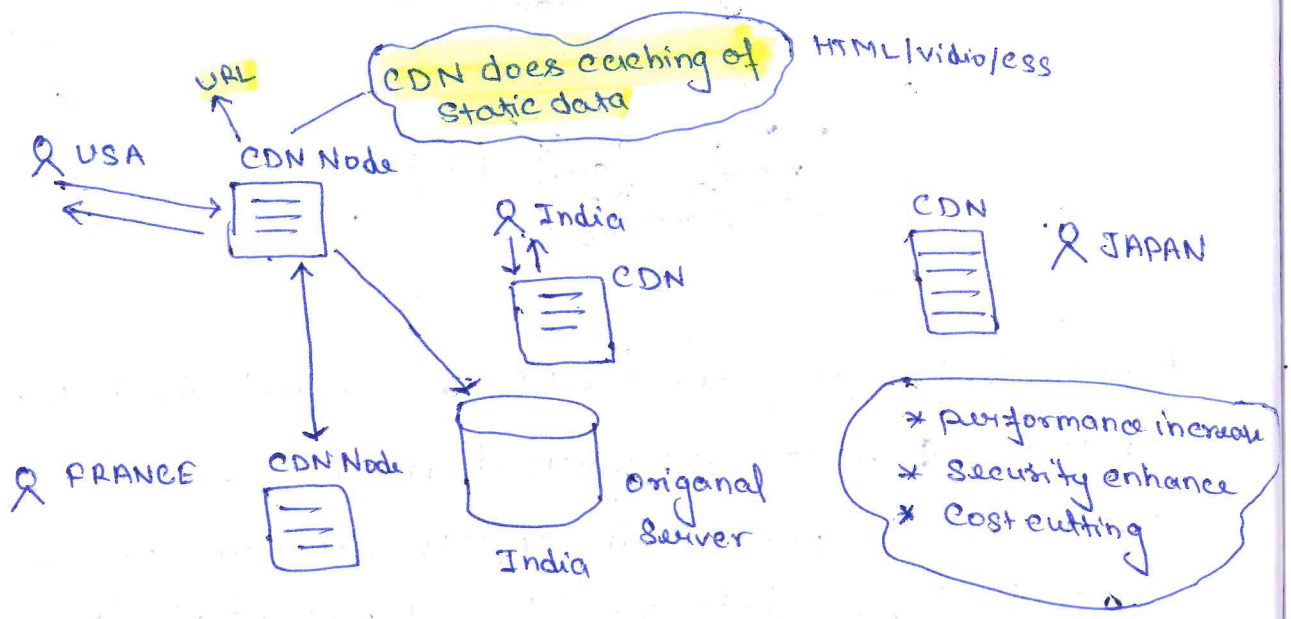
# 6. CDN (Content Delivery Network)

* - CDN does CACHING.
But
All those whose does caching are Not CDN.



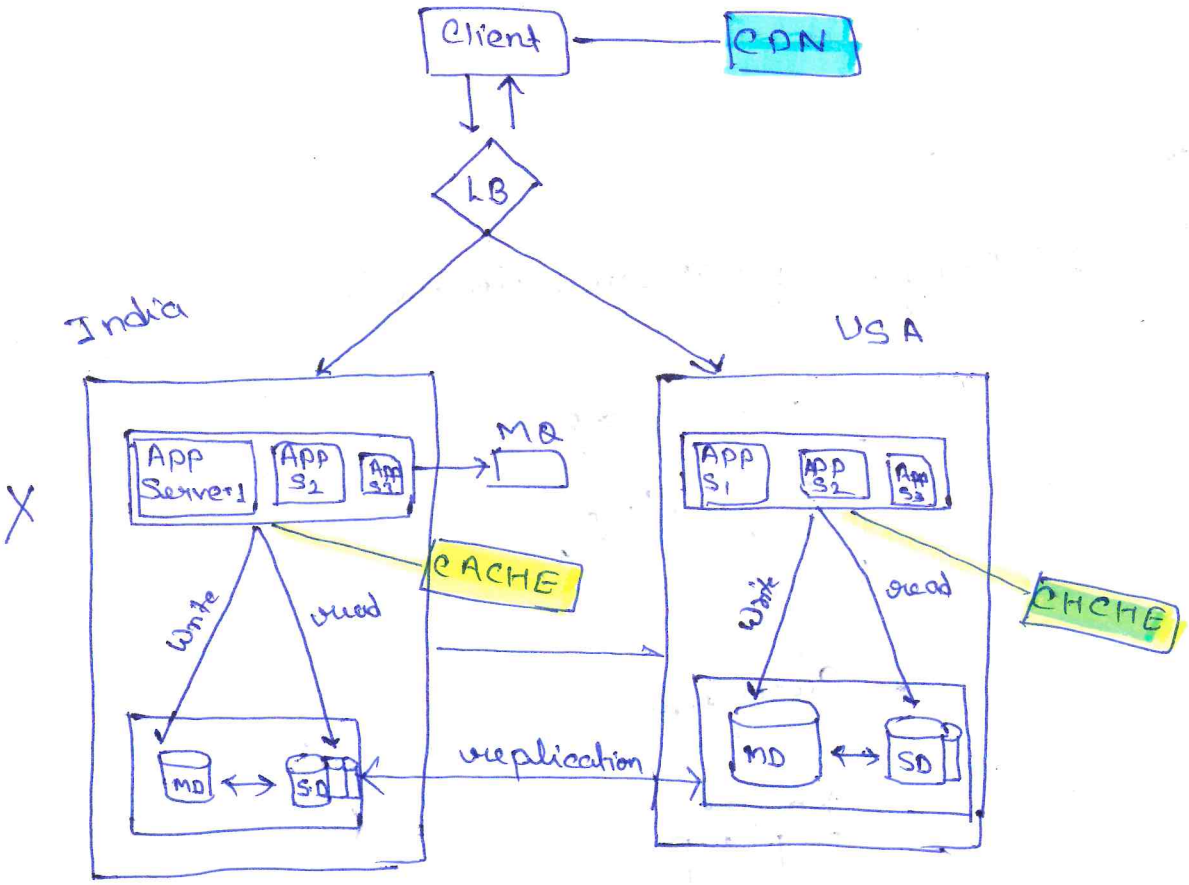

Concept && Coding

That time Latency issue. This probleam can solve CDN




* User request firstly go on ~~ma~~ own CDN. If data is present It is OK. If data is not present it's go on nearest CDN. If nearest CDN Data is not present then it's go on Origanal Server.

## 7. Data Centre :→



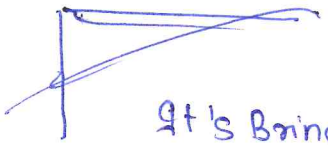

I have a advantages

Concept && Coding

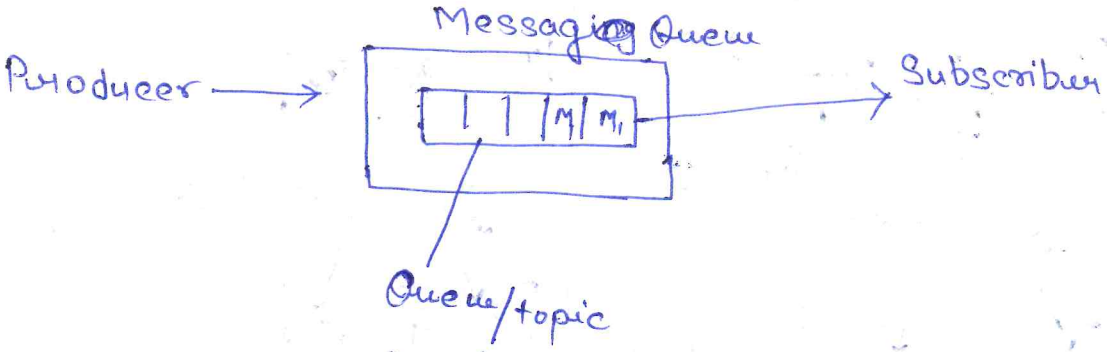
- If India DataCentre is down, the LoadBalancer can full fill the requirement for USA Server.
- That time I have one problem. If IndiaServer is down we have full fill the requirement from USA Server. that time only USA data will found.

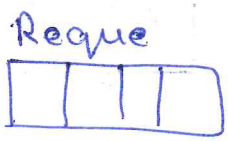
Solution is replication

## 8. Messaging Queue :- RabitMq, Kafka



It's Bring a ASCYNature in codebase

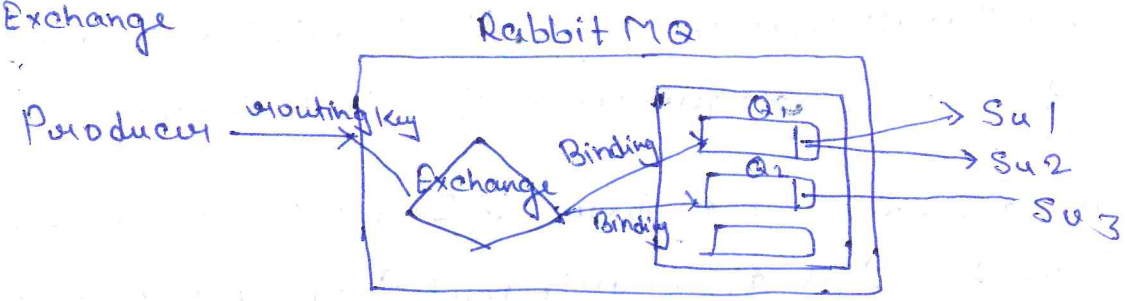


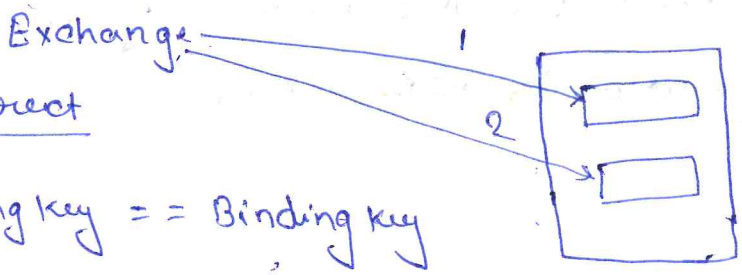




## Termology of Messiging Queue

Concept && Coding

Exchange



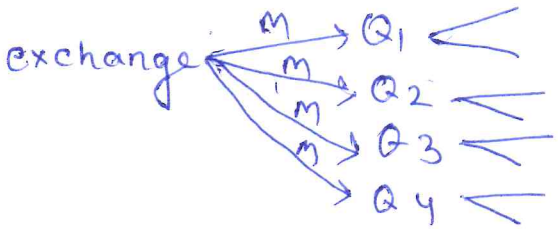




① Direct

routing key == Binding key

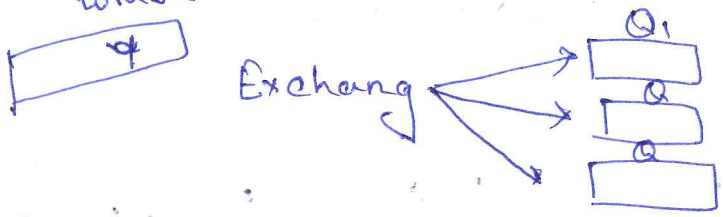
1

② FAN OUT



3 Topic

wild Com



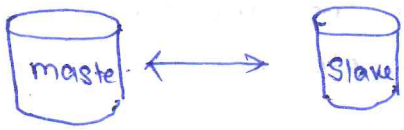

## 9. DataBase Scaling :→

Database scaling is two type :

1. Vertical
2. Horizantal

### 1 Vertical :→

Concept && Coding




→ Vertical Scalling tell that already DB Server you can increase capcity. you can increase CPU and RAM capcity. Because comes more request CPU and RAM takecare.

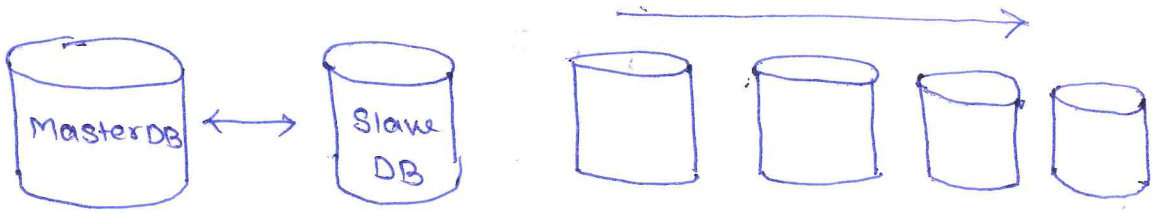
CPU ↑ Increase
RAM

Disadvantages :-

1 There is always a limit. we cannot increase ~~after a point~~. capbility of CPU and RAM after a point.

## 2:- Horizantal :-

Horizantal tell us you can add more nodes



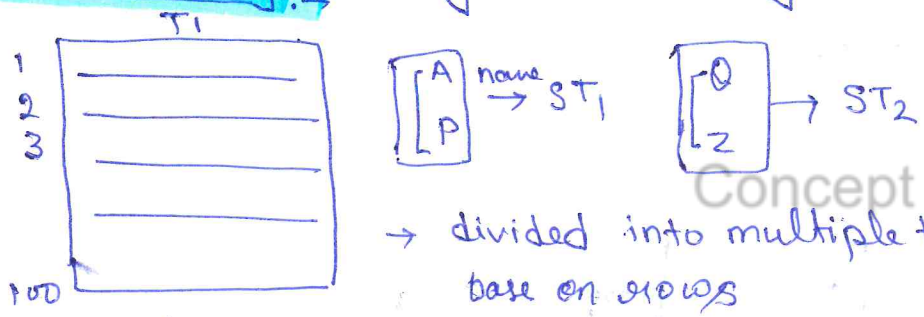

Implementation

SHARDING

① Horizantal Sharding

② Vertical Sharding

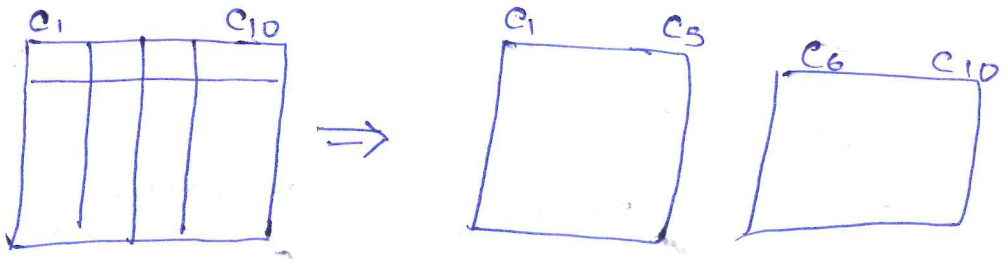
### 1 Horizantal Sharding :→




→ divided into multiple table base on rows

Concept && Coding

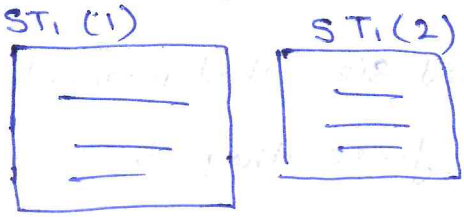
2. **Vertical Sharding** :- divided ~~into~~ into multiple table based on column.




## Drawbacks of Sharding :→

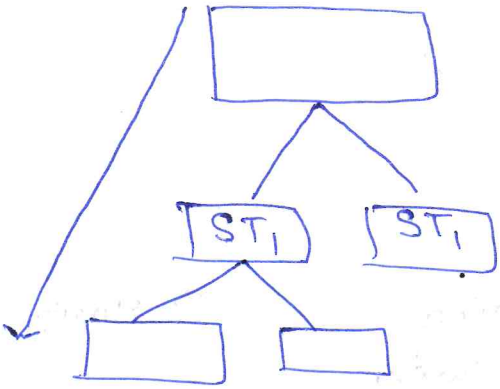
For Example

(1) Let say that A set is very large, we have A top is going on ~~A top~~ ST1. that time ST1 is imedity fill up. Then we again shard.




* In Sharding make Tree possible.

Concept && Coding




* Tree can solve with consistence Hashing

(2) Suppose rows are divided. If we want to JOIN. We can't join. We can solve we denormilization.